

HubSpot Workflow Automation - End-to-End Documentation

Muhammad JAMEEL

1 System Overview

- **Purpose:** Automate HubSpot lead management through natural language commands
- **Architecture:** Multi-agent system with specialized agents for different functions
- **Technology:** OpenAI Assistants API + HubSpot API + Email services

2 Core Components

2.1 Configuration (`config.json` + `utils/config_loader.py`)

- Stores all API credentials and service endpoints
- Loads configuration from file or environment variables
- Validates necessary credentials are present

2.2 Agent Structure

2.2.1 Orchestrator Agent (`agents/orchestrator.py`)

- Creates and manages OpenAI Assistant with function tools
- Processes natural language queries
- Routes function calls to appropriate sub-agents
- Coordinates workflow between agents
- Returns final results to user

2.2.2 HubSpot Agent (`agents/hubspot_agent.py`)

- Communicates with HubSpot API
- Functions:
 - `check_lead`: Searches for leads by email
 - `create_lead`: Creates new leads in HubSpot
 - `update_lead`: Updates existing lead records
 - `get_all_leads`: Retrieves multiple leads

2.2.3 Email Agent (`agents/email_agent.py`)

- Manages email notifications
- Functions:
 - `send_notification`: Sends emails via SMTP
 - `create_lead_notification`: Formats lead creation emails
 - `create_update_notification`: Formats lead update emails

2.3 Error Handling (`utils/error_handler.py`)

- Custom exceptions for each agent type
- Decorator pattern for consistent error management
- Logging of all errors with traceability

2.4 Main Application (`app.py`)

- Entry point for the application
- Argument parsing for different run modes
- Sets up the agent system
- Provides interactive or single-query interfaces

3 Query Processing Flow

1. Query Submission

- User submits natural language query via CLI
- Query is passed to the Orchestrator

2. Query Understanding

- Orchestrator uses OpenAI Assistant to understand intent
- Assistant identifies required actions based on query

3. Function Calling

- Assistant determines which functions to call with what parameters
- Orchestrator receives function call requests

4. Task Delegation

- Orchestrator routes function calls to appropriate agents
- HubSpot agent performs CRM operations
- Email agent sends notifications after successful actions

5. Result Collection

- Orchestrator collects results from all agent operations
- Results are formatted and returned to Assistant

6. Response Generation

- Assistant generates natural language response based on action results
- Response is displayed to user

4 Example Query Flow: "Create a lead for john@example.com named John Smith"

1. User enters query in `app.py` interactive mode
2. `OrchestratorAgent.process_query()` sends query to OpenAI Assistant
3. Assistant determines intent is to create a lead
4. Assistant requests `create_lead` function call with parameters:

```
{ "email": "john@example.com", "firstname": "John", "lastname": "Smith" }
```
5. Orchestrator routes request to `HubspotAgent.create_lead()`
6. HubSpot agent makes API call to create lead in CRM
7. On success, orchestrator calls `EmailAgent.create_lead_notification()`
8. Email agent formats notification and sends email
9. Results from both operations return to Assistant
10. Assistant generates confirmation message
11. Message displayed to user in CLI

5 Error Handling Workflow

1. If any API call fails:
 - Exception is caught by `handle_agent_errors` decorator
 - Error is logged with details
 - Formatted error response is returned
 - Orchestrator receives error and passes to Assistant
 - Assistant generates appropriate error message for user

6 Command Reference

- **Interactive Mode:** `python app.py --interactive`
- **Single Query:** `python app.py --query "Your query here"`
- **Custom Config:** `python app.py --config custom_config.json --interactive`